Virtual Lab Experiment-6

Jenkins with Build Tool (Maven)

Aim

To **implement Continuous Integration using Jenkins and Maven** where source code is automatically built and tested whenever changes are pushed to a Git repository.

Scenario (Real-Time Use Case)

A software company wants to **automatically build and test a Java application whenever developers push code to GitHub**.

The requirements are:

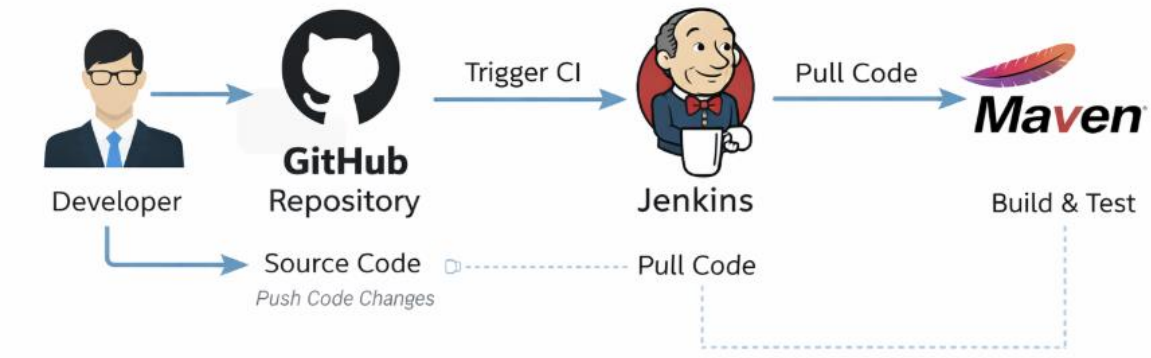
- Source code should be stored in Git
- Build process should be automated
- Build tool should compile the application
- Jenkins should trigger build automatically

Solution Stack

Requirement	Tool
Version Control	Git
CI/CD Tool	Jenkins
Build Tool	Maven
Programming Language	Java
Repository	GitHub
Operating System	Linux / Ubuntu

Architecture Overview

1. Developer pushes code to GitHub
2. Jenkins detects the code change
3. Jenkins pulls the source code
4. Maven compiles the application
5. Jenkins shows build status



Tools & Technologies

- Git
- GitHub
- Jenkins
- Maven
- Java JDK
- Linux / Ubuntu

System Requirements

- OS: Ubuntu / Linux / Windows
- RAM: Minimum 4 GB
- Java JDK installed
- Git installed
- Maven installed
- Jenkins installed

Project Directory Structure

```
jenkins-lab/
│
├── src/
│   └── HelloWorld.java
│
└── pom.xml
```

Step-by-Step Implementation

PART A – Install Required Tools

Step 1: Install Java

```
sudo apt update
sudo apt install openjdk-11-jdk -y
```

Check version

```
java -version
```

Step 2: Install Maven

```
sudo apt install maven -y
```

Check version

```
mvn -version
```

Step 3: Install Jenkins

```
sudo apt install jenkins -y
```

Start Jenkins

```
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

Check status

```
sudo systemctl status jenkins
```

Step 4: Access Jenkins

Open browser

http://localhost:8080

Unlock Jenkins

sudo cat /var/lib/jenkins/secrets/initialAdminPassword

Install **Suggested Plugins**

PART B – Create Sample Java Application

Step 5: Create Java Program

Create directory

mkdir jenkins-lab

cd jenkins-lab

mkdir src

Create file

nano src/HelloWorld.java

Example Code

```
public class HelloWorld {
   public static void main(String[] args) {
      System.out.println("Hello from Jenkins CI Pipeline!");
   }
}
```

PART C – Create Maven Build File

Step 6: Create pom.xml

nano pom.xml

Example

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
 <modelVersion>4.0.0</modelVersion>

 <groupId>com.devops</groupId>
 <artifactId>jenkins-lab</artifactId>
 <version>1.0</version>

 <build>
  <sourceDirectory>src</sourceDirectory>
  <plugins>
   <plugin>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>3.8.1</version>
   </plugin>
  </plugins>
 </build>
</project>
```

PART D – Push Code to GitHub

Initialize repository

git init

git add .

git commit -m "Initial Jenkins Maven Project"

Add GitHub repository

git remote add origin <repo-url>

git push origin main

PART E – Configure Jenkins Job

Step 7: Create Jenkins Job

1. Open Jenkins Dashboard
2. Click **New Item**
3. Enter job name
4. Select **Freestyle Project**
5. Click **OK**

Step 8: Configure Source Code

In **Source Code Management**

Select **Git**

Enter repository URL

https://github.com/username/jenkins-lab.git

Branch

*/main

Step 9: Configure Build Tool

In **Build Section**

Select

Invoke top-level Maven targets

Goals

clean package

PART F – Build the Project

Click

Build Now

Jenkins will

- Pull source code
- Run Maven build
- Compile Java application

Check **Console Output**

Expected Output

BUILD SUCCESS

Observations

- Jenkins successfully integrated with GitHub
- Maven automatically compiled the Java project
- Jenkins dashboard shows build status

Result

The **Continuous Integration pipeline was successfully implemented using Jenkins and Maven**, where Jenkins automatically builds the application whenever source code is updated.

VIVA VOCE QUESTIONS

Basic

1. What is Jenkins?
2. What is Continuous Integration?
3. What is Maven?
4. What is a build tool?
5. What is pom.xml?

Intermediate

6. Difference between CI and CD
7. What are Jenkins plugins?
8. What is a Jenkins pipeline?
9. What is a workspace in Jenkins?
10. Difference between freestyle job and pipeline job

Advanced

11. How does Jenkins trigger builds automatically?
12. What is webhook in Jenkins?
13. How does Maven manage dependencies?
14. What is artifact generation in Maven?
15. Explain Jenkins pipeline stages.
